



Product Performance Analysis – SQL Script Pack

Below are the complete scripts for the Product Performance Analysis project. Each script mirrors the structure of the reference pack while implementing the logic for the product performance domain.

Complete SQL Scripts

Below are the complete scripts for the Product Performance Analysis project. Each script mirrors the structure of the reference pack while implementing the logic for the product performance domain.

 **Important:** All scripts are provided in the input document and should be executed in the order specified in the runbook. Each script includes comprehensive comments explaining the what, why and how of each step.

The scripts implement:

- Database and schema setup (00_setup_db_and_schemas.sql)
- Bronze layer table creation (01_create_bronze_tables.sql)
- Synthetic data generation (02_insert_bronze_data_FULL.sql)
- Silver layer transformation (03_silver_clean_transform.sql)
- Gold star schema (04_gold_star_schema.sql)
- Presentation views (05_pres_views.sql)
- Performance metrics (06_readmission_metrics.sql)
- ETL stored procedures (07_etl_stored_procs.sql)
- Validation and reconciliation (08_validation_reconciliation.sql)
- SQL fundamentals training (09_training_sql_fundamentals.sql)

00_setup_db_and_schemas.sql

```
/*
- Script 00 – Setup Database and Schemas
- Drops any existing TeamProject_ProductPerformance database, creates a fresh database
and defines the schemas required for the Medallion architecture. Run this
script once at the beginning of the project. */

IF DB_ID('TeamProject_ProductPerformance') IS NOT NULL
BEGIN
    ALTER DATABASE TeamProject_ProductPerformance SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
    DROP DATABASE TeamProject_ProductPerformance;
END
GO

CREATE DATABASE TeamProject_ProductPerformance;
GO

USE TeamProject_ProductPerformance;
GO

-- Create schemas representing Medallion layers and supporting schemas
CREATE SCHEMA bronze;
CREATE SCHEMA silver;
CREATE SCHEMA gold;
CREATE SCHEMA pres;
CREATE SCHEMA etl;
CREATE SCHEMA dq;
GO

-- Validation: confirm schemas exist
SELECT name AS SchemaName
FROM sys.schemas
WHERE name IN ('bronze','silver','gold','pres','etl','dq');
GO
```

01_create_bronze_tables.sql

```
/* 01_create_bronze_tables.sql
- Creates Bronze raw tables (minimal constraints)
- Creates dq logging table */

USE TeamProject_ProductPerformance;
GO

-- Drop in dependency-safe order
IF OBJECT_ID('bronze.Sales','U') IS NOT NULL DROP TABLE bronze.Sales;
IF OBJECT_ID('bronze.Customer','U') IS NOT NULL DROP TABLE bronze.Customer;
IF OBJECT_ID('bronze.Product','U') IS NOT NULL DROP TABLE bronze.Product;
IF OBJECT_ID('bronze.Category','U') IS NOT NULL DROP TABLE bronze.Category;
IF OBJECT_ID('bronze.Region','U') IS NOT NULL DROP TABLE bronze.Region;
GO

IF OBJECT_ID('dq.DataQualityLog','U') IS NOT NULL DROP TABLE dq.DataQualityLog;
GO

-- Category (raw)
CREATE TABLE bronze.Category (
  CategoryID INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_bronze_Category PRIMARY KEY,
  CategoryCode NVARCHAR(20) NOT NULL,
  CategoryName NVARCHAR(100) NOT NULL,
  CreatedDTTM DATETIME2(0) NOT NULL CONSTRAINT DF_bronze_Category_Created DEFAULT
SYSUTCDATETIME()
);

-- Product (raw)
CREATE TABLE bronze.Product (
  ProductID INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_bronze_Product PRIMARY KEY,
  CategoryID INT NULL,
  ProductCode NVARCHAR(20) NOT NULL,
  ProductName NVARCHAR(100) NOT NULL,
  CreatedDTTM DATETIME2(0) NOT NULL CONSTRAINT DF_bronze_Product_Created DEFAULT
SYSUTCDATETIME()
);

-- Region (raw)
CREATE TABLE bronze.Region (
  RegionID INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_bronze_Region PRIMARY KEY,
  RegionCode NVARCHAR(10) NOT NULL,
  RegionName NVARCHAR(100) NOT NULL,
  CreatedDTTM DATETIME2(0) NOT NULL CONSTRAINT DF_bronze_Region_Created DEFAULT
SYSUTCDATETIME()
);

-- Customer (raw)
CREATE TABLE bronze.Customer (
  CustomerID INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_bronze_Customer PRIMARY KEY,
  RegionID INT NULL,
  CustomerCode NVARCHAR(25) NOT NULL,
  CustomerName NVARCHAR(100) NOT NULL,
  CreatedDTTM DATETIME2(0) NOT NULL CONSTRAINT DF_bronze_Customer_Created DEFAULT
SYSUTCDATETIME()
);

-- Sales (raw)
CREATE TABLE bronze.Sales (
  SalesID INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_bronze_Sales PRIMARY KEY,
  ProductID INT NULL,
  RegionID INT NULL,
  CustomerID INT NULL,
  SalesDateTime DATETIME2(0) NOT NULL,
  Quantity INT NOT NULL,
  UnitPrice DECIMAL(10,2) NOT NULL,
  CreatedDTTM DATETIME2(0) NOT NULL CONSTRAINT DF_bronze_Sales_Created DEFAULT
SYSUTCDATETIME()
);

-- Data Quality log
CREATE TABLE dq.DataQualityLog (
  LogID INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_dq_DataQualityLog PRIMARY KEY,
  TableName NVARCHAR(128) NOT NULL,
  CheckName NVARCHAR(256) NOT NULL,
  CheckResult NVARCHAR(256) NOT NULL,
  ErrorCount INT NOT NULL,
  RunDTTM DATETIME2(0) NOT NULL CONSTRAINT DF_dq_Run DEFAULT SYSUTCDATETIME()
);
GO

-- Validation: list bronze tables
SELECT TABLE_NAME
FROM INFORMATION_SCHEMA.TABLES
WHERE TABLE_SCHEMA = 'bronze';
GO
```

02_insert_bronze_data_FULL.sql

```
/* 02_insert_bronze_data_FULL.sql FIXED VERSION – NO CROSS-BATCH VARIABLES
Generates synthetic data for the product performance domain. */

USE TeamProject_ProductPerformance;
GO
SET NOCOUNT ON;

-- CLEAR EXISTING DATA
DELETE FROM bronze.Sales;
DELETE FROM bronze.Customer;
DELETE FROM bronze.Product;
DELETE FROM bronze.Category;
DELETE FROM bronze.Region;
GO

-- TALLY TABLE (100,000 rows)
IF OBJECT_ID('tempdb..#Numbers') IS NOT NULL DROP TABLE #Numbers;

;WITH N AS (
  SELECT TOP (100000) ROW_NUMBER() OVER (ORDER BY (SELECT NULL)) AS n
  FROM sys.all_objects a
  CROSS JOIN sys.all_objects b
)
SELECT n INTO #Numbers FROM N;
GO

-- REGIONS (5)
BEGIN TRAN;
INSERT INTO bronze.Region (RegionCode, RegionName)
VALUES ('R001','North'),
('R002','South'),
('R003','East'),
('R004','West'),
('R005','Central');
COMMIT;
GO

-- CATEGORIES (10)
BEGIN TRAN;
INSERT INTO bronze.Category (CategoryCode, CategoryName)
SELECT
  CONCAT('C', RIGHT('000' + CAST(n AS VARCHAR(3)), 3)) AS CategoryCode,
  CONCAT('Category ', RIGHT('000' + CAST(n AS VARCHAR(3)), 3)) AS CategoryName
FROM #Numbers
WHERE n <= 10;
COMMIT;
GO

-- PRODUCTS (100)
BEGIN TRAN;
INSERT INTO bronze.Product (CategoryID, ProductCode, ProductName)
SELECT
  ((n - 1) % 10) + 1 AS CategoryID,
  CONCAT('P', RIGHT('0000' + CAST(n AS VARCHAR(4)), 4)) AS ProductCode,
  CONCAT('Product_', RIGHT('0000' + CAST(n AS VARCHAR(4)), 4)) AS ProductName
FROM #Numbers
WHERE n <= 100;
COMMIT;
GO

-- CUSTOMERS (2,000)
BEGIN TRAN;
INSERT INTO bronze.Customer (RegionID, CustomerCode, CustomerName)
SELECT
  ((n - 1) % 5) + 1 AS RegionID,
  CONCAT('CUS', RIGHT('0000' + CAST(n AS VARCHAR(4)), 4)) AS CustomerCode,
  CONCAT('Customer_', RIGHT('0000' + CAST(n AS VARCHAR(4)), 4)) AS CustomerName
FROM #Numbers
WHERE n <= 2000;
COMMIT;
GO

-- SALES (50,000)
BEGIN TRAN;
INSERT INTO bronze.Sales (
  ProductID,
  RegionID,
  CustomerID,
  SalesDateTime,
  Quantity,
  UnitPrice
)
SELECT
  1 + (ABS(CHECKSUM(n*17)) % 100) AS ProductID,
  1 + (ABS(CHECKSUM(n*19)) % 5) AS RegionID,
  1 + (ABS(CHECKSUM(n*23)) % 2000) AS CustomerID,
  DATEADD(DAY, -1 * (ABS(CHECKSUM(n*29)) % 730), '2025-01-01') AS SalesDateTime,
  -- Introduce occasional zero or negative quantities for DQ testing
  CASE
    WHEN n % 100 = 0 THEN -1
    WHEN n % 50 = 0 THEN 0
    ELSE 1 + (ABS(CHECKSUM(n*31)) % 5)
  END AS Quantity,
  -- Introduce occasional zero or negative unit prices for DQ testing
  CASE
    WHEN n % 200 = 0 THEN -5.00
    WHEN n % 75 = 0 THEN 0.00
    ELSE 5.00 + (ABS(CHECKSUM(n*37)) % 100)
  END AS UnitPrice
FROM #Numbers
WHERE n <= 50000;
COMMIT;
GO

DROP TABLE #Numbers;
GO

-- Validation: counts
SELECT 'Category' AS Entity, (SELECT COUNT(*) FROM bronze.Category) AS RowCount
UNION ALL
SELECT 'Product', (SELECT COUNT(*) FROM bronze.Product)
UNION ALL
SELECT 'Region', (SELECT COUNT(*) FROM bronze.Region)
UNION ALL
SELECT 'Customer', (SELECT COUNT(*) FROM bronze.Customer)
UNION ALL
SELECT 'Sales', (SELECT COUNT(*) FROM bronze.Sales);
GO
```


03_silver_clean_transform.sql

```
/* 03_silver_clean_transform.sql
- Creates Silver tables (cleaned/standardised + constraints)
- Uses proper key mapping tables (no correlated subqueries)
- Preserves raw SalesID via BronzeSalesID for stable mapping
- Logs DQ counts */

USE TeamProject_ProductPerformance;
GO

SET NOCOUNT ON;

-- Drop Silver tables in dependency order
IF OBJECT_ID('silver.Sales','U') IS NOT NULL DROP TABLE silver.Sales;
IF OBJECT_ID('silver.Customer','U') IS NOT NULL DROP TABLE silver.Customer;
IF OBJECT_ID('silver.Product','U') IS NOT NULL DROP TABLE silver.Product;
IF OBJECT_ID('silver.Category','U') IS NOT NULL DROP TABLE silver.Category;
IF OBJECT_ID('silver.Region','U') IS NOT NULL DROP TABLE silver.Region;
GO

-- Silver: Category
CREATE TABLE silver.Category (
    CategoryKey INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_silver_Category PRIMARY KEY,
    CategoryCode NVARCHAR(20) NOT NULL,
    CategoryName NVARCHAR(100) NOT NULL,
    CONSTRAINT UQ_silver_Category UNIQUE (CategoryCode)
);

-- Silver: Region
CREATE TABLE silver.Region (
    RegionKey INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_silver_Region PRIMARY KEY,
    RegionCode NVARCHAR(10) NOT NULL,
    RegionName NVARCHAR(100) NOT NULL,
    CONSTRAINT UQ_silver_Region UNIQUE (RegionCode)
);

-- Silver: Product
CREATE TABLE silver.Product (
    ProductKey INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_silver_Product PRIMARY KEY,
    CategoryKey INT NOT NULL,
    ProductCode NVARCHAR(20) NOT NULL,
    ProductName NVARCHAR(100) NOT NULL,
    CONSTRAINT UQ_silver_Product UNIQUE (ProductCode),
    CONSTRAINT FK_silver_Product_Category FOREIGN KEY (CategoryKey) REFERENCES
silver.Category(CategoryKey)
);

-- Silver: Customer
CREATE TABLE silver.Customer (
    CustomerKey INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_silver_Customer PRIMARY KEY,
    RegionKey INT NOT NULL,
    CustomerCode NVARCHAR(25) NOT NULL,
    CustomerName NVARCHAR(100) NOT NULL,
    CONSTRAINT UQ_silver_Customer UNIQUE (CustomerCode),
    CONSTRAINT FK_silver_Customer_Region FOREIGN KEY (RegionKey) REFERENCES silver.Region(RegionKey)
);

-- Silver: Sales (preserve BronzeSalesID)
CREATE TABLE silver.Sales (
    SalesKey INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_silver_Sales PRIMARY KEY,
    BronzeSalesID INT NOT NULL,
    ProductKey INT NOT NULL,
    RegionKey INT NOT NULL,
    CustomerKey INT NOT NULL,
    SalesDateTime DATETIME2(0) NOT NULL,
    Quantity INT NOT NULL,
    UnitPrice DECIMAL(10,2) NOT NULL,
    SalesAmount AS (Quantity * UnitPrice) PERSISTED,
    CONSTRAINT UQ_silver_Sales_Bronze UNIQUE (BronzeSalesID),
    CONSTRAINT FK_silver_Sales_Product FOREIGN KEY (ProductKey) REFERENCES silver.Product(ProductKey),
    CONSTRAINT FK_silver_Sales_Region FOREIGN KEY (RegionKey) REFERENCES silver.Region(RegionKey),
    CONSTRAINT FK_silver_Sales_Customer FOREIGN KEY (CustomerKey) REFERENCES
silver.Customer(CustomerKey)
);
GO

/* ===== LOAD SILVER (SET-BASED) ===== */

-- 1) Categories
INSERT INTO silver.Category (CategoryCode, CategoryName)
SELECT DISTINCT
    CategoryCode,
    CONCAT(UPPER(LEFT(LTRIM(RTRIM(CategoryName)),1)),
    LOWER(SUBSTRING(LTRIM(RTRIM(CategoryName)),2,200))) AS CategoryName
FROM bronze.Category;

-- Mapping table for Category
IF OBJECT_ID('tempdb..#CategoryMap') IS NOT NULL DROP TABLE #CategoryMap;
SELECT b.CategoryID, s.CategoryKey
INTO #CategoryMap
FROM bronze.Category b
JOIN silver.Category s ON s.CategoryCode = b.CategoryCode;

-- 2) Regions
INSERT INTO silver.Region (RegionCode, RegionName)
SELECT DISTINCT
    RegionCode,
    CONCAT(UPPER(LEFT(LTRIM(RTRIM(RegionName)),1)),
    LOWER(SUBSTRING(LTRIM(RTRIM(RegionName)),2,200))) AS RegionName
FROM bronze.Region;

-- Mapping table for Region
IF OBJECT_ID('tempdb..#RegionMap') IS NOT NULL DROP TABLE #RegionMap;
SELECT b.RegionID, s.RegionKey
INTO #RegionMap
FROM bronze.Region b
JOIN silver.Region s ON s.RegionCode = b.RegionCode;

-- 3) Products (dedupe by ProductCode; assign CategoryKey)
;WITH SrcProd AS (
    SELECT
        b.ProductID,
        b.CategoryID,
        b.ProductCode,
        b.ProductName,
        ROW_NUMBER() OVER (PARTITION BY b.ProductCode ORDER BY b.ProductID) AS rn
    FROM bronze.Product b
)
INSERT INTO silver.Product (CategoryKey, ProductCode, ProductName)
SELECT
    cm.CategoryKey,
    sp.ProductCode,
    CONCAT(UPPER(LEFT(LTRIM(RTRIM(sp.ProductName)),1)),
    LOWER(SUBSTRING(LTRIM(RTRIM(sp.ProductName)),2,200))) AS ProductName
FROM SrcProd sp
JOIN #CategoryMap cm ON cm.CategoryID = sp.CategoryID
WHERE sp.rn = 1;

-- Mapping table for Product
IF OBJECT_ID('tempdb..#ProductMap') IS NOT NULL DROP TABLE #ProductMap;
SELECT b.ProductID, s.ProductKey
INTO #ProductMap
FROM bronze.Product b
JOIN silver.Product s ON s.ProductCode = b.ProductCode;

-- 4) Customers (dedupe by CustomerCode; assign RegionKey)
;WITH SrcCust AS (
    SELECT
        b.CustomerID,
        b.RegionID,
        b.CustomerCode,
        b.CustomerName,
        ROW_NUMBER() OVER (PARTITION BY b.CustomerCode ORDER BY b.CustomerID) AS rn
    FROM bronze.Customer b
)
INSERT INTO silver.Customer (RegionKey, CustomerCode, CustomerName)
SELECT
    rm.RegionKey,
    sc.CustomerCode,
    CONCAT(UPPER(LEFT(LTRIM(RTRIM(sc.CustomerName)),1)),
    LOWER(SUBSTRING(LTRIM(RTRIM(sc.CustomerName)),2,200))) AS CustomerName
FROM SrcCust sc
JOIN #RegionMap rm ON rm.RegionID = sc.RegionID
WHERE sc.rn = 1;

-- Mapping table for Customer
IF OBJECT_ID('tempdb..#CustomerMap') IS NOT NULL DROP TABLE #CustomerMap;
SELECT b.CustomerID, s.CustomerKey
INTO #CustomerMap
FROM bronze.Customer b
JOIN silver.Customer s ON s.CustomerCode = b.CustomerCode;

-- 5) Sales (cleanse negative/zero quantity and unit price)
INSERT INTO silver.Sales (
    BronzeSalesID,
    ProductKey,
    RegionKey,
    CustomerKey,
    SalesDateTime,
    Quantity,
    UnitPrice
)
SELECT
    bs.SalesID AS BronzeSalesID,
    pm.ProductKey,
    rm.RegionKey,
    cm.CustomerKey,
    bs.SalesDateTime,
    CASE WHEN bs.Quantity <= 0 THEN 0 ELSE bs.Quantity END AS Quantity,
    CASE WHEN bs.UnitPrice <= 0 THEN 0.00 ELSE bs.UnitPrice END AS UnitPrice
FROM bronze.Sales bs
JOIN #ProductMap pm ON pm.ProductID = bs.ProductID
JOIN #RegionMap rm ON rm.RegionID = bs.RegionID
JOIN #CustomerMap cm ON cm.CustomerID = bs.CustomerID;

-- DQ logs: invalid quantity or unit price
INSERT INTO dq.DataQualityLog (TableName, CheckName, CheckResult, ErrorCount)
SELECT
    'bronze.Sales',
    'Non-positive quantity or unit price',
    'Capped at zero in Silver',
    COUNT(*)
FROM bronze.Sales
WHERE Quantity <= 0 OR UnitPrice <= 0;
GO

-- Validation: row counts
SELECT 'Category' AS Entity,
    (SELECT COUNT(*) FROM bronze.Category) AS BronzeCount,
    (SELECT COUNT(*) FROM silver.Category) AS SilverCount
UNION ALL
SELECT 'Product',
    (SELECT COUNT(*) FROM bronze.Product),
    (SELECT COUNT(*) FROM silver.Product)
UNION ALL
SELECT 'Region',
    (SELECT COUNT(*) FROM bronze.Region),
    (SELECT COUNT(*) FROM silver.Region)
UNION ALL
SELECT 'Customer',
    (SELECT COUNT(*) FROM bronze.Customer),
    (SELECT COUNT(*) FROM silver.Customer)
UNION ALL
SELECT 'Sales',
    (SELECT COUNT(*) FROM bronze.Sales),
    (SELECT COUNT(*) FROM silver.Sales);
GO
```


04_gold_star_schema.sql

```
/* 04_gold_star_schema.sql
- Builds Gold star schema
- Dimensions: Date, Product, Category, Region, Customer
- FactSales is one row per sales transaction (grain) */

USE TeamProject_ProductPerformance;
GO

SET NOCOUNT ON;

-- Drop facts first
IF OBJECT_ID('gold.FactSales','U') IS NOT NULL DROP TABLE gold.FactSales;

-- Drop dimensions
IF OBJECT_ID('gold.DimCustomer','U') IS NOT NULL DROP TABLE gold.DimCustomer;
IF OBJECT_ID('gold.DimRegion','U') IS NOT NULL DROP TABLE gold.DimRegion;
IF OBJECT_ID('gold.DimProduct','U') IS NOT NULL DROP TABLE gold.DimProduct;
IF OBJECT_ID('gold.DimCategory','U') IS NOT NULL DROP TABLE gold.DimCategory;
IF OBJECT_ID('gold.DimDate','U') IS NOT NULL DROP TABLE gold.DimDate;
GO

-- Dimensions
CREATE TABLE gold.DimDate (
    DateKey INT NOT NULL CONSTRAINT PK_gold_DimDate PRIMARY KEY,
    [Date] DATE NOT NULL,
    [Day] INT NOT NULL,
    [Month] INT NOT NULL,
    MonthName NVARCHAR(20) NOT NULL,
    [Quarter] INT NOT NULL,
    [Year] INT NOT NULL,
    IsWeekend BIT NOT NULL
);

CREATE TABLE gold.DimCategory (
    CategoryKey INT NOT NULL CONSTRAINT PK_gold_DimCategory PRIMARY KEY,
    CategoryCode NVARCHAR(20) NOT NULL,
    CategoryName NVARCHAR(100) NOT NULL
);

CREATE TABLE gold.DimRegion (
    RegionKey INT NOT NULL CONSTRAINT PK_gold_DimRegion PRIMARY KEY,
    RegionCode NVARCHAR(10) NOT NULL,
    RegionName NVARCHAR(100) NOT NULL
);

CREATE TABLE gold.DimProduct (
    ProductKey INT NOT NULL CONSTRAINT PK_gold_DimProduct PRIMARY KEY,
    CategoryKey INT NOT NULL,
    ProductCode NVARCHAR(20) NOT NULL,
    ProductName NVARCHAR(100) NOT NULL,
    CONSTRAINT FK_gold_DimProduct_Category FOREIGN KEY (CategoryKey) REFERENCES
gold.DimCategory(CategoryKey)
);

CREATE TABLE gold.DimCustomer (
    CustomerKey INT NOT NULL CONSTRAINT PK_gold_DimCustomer PRIMARY KEY,
    RegionKey INT NOT NULL,
    CustomerCode NVARCHAR(25) NOT NULL,
    CustomerName NVARCHAR(100) NOT NULL,
    CONSTRAINT FK_gold_DimCustomer_Region FOREIGN KEY (RegionKey) REFERENCES
gold.DimRegion(RegionKey)
);

-- Facts
CREATE TABLE gold.FactSales (
    SalesKey INT NOT NULL CONSTRAINT PK_gold_FactSales PRIMARY KEY,
    ProductKey INT NOT NULL,
    RegionKey INT NOT NULL,
    CustomerKey INT NOT NULL,
    DateKey INT NOT NULL,
    Quantity INT NOT NULL,
    UnitPrice DECIMAL(10,2) NOT NULL,
    SalesAmount DECIMAL(18,2) NOT NULL,
    CONSTRAINT FK_gold_FS_Product FOREIGN KEY (ProductKey) REFERENCES gold.DimProduct(ProductKey),
    CONSTRAINT FK_gold_FS_Region FOREIGN KEY (RegionKey) REFERENCES gold.DimRegion(RegionKey),
    CONSTRAINT FK_gold_FS_Customer FOREIGN KEY (CustomerKey) REFERENCES
gold.DimCustomer(CustomerKey),
    CONSTRAINT FK_gold_FS_Date FOREIGN KEY (DateKey) REFERENCES gold.DimDate(DateKey)
);
GO

/* ===== LOAD GOLD DIMENSIONS ===== */

-- Category
INSERT INTO gold.DimCategory (CategoryKey, CategoryCode, CategoryName)
SELECT CategoryKey, CategoryCode, CategoryName
FROM silver.Category;

-- Region
INSERT INTO gold.DimRegion (RegionKey, RegionCode, RegionName)
SELECT RegionKey, RegionCode, RegionName
FROM silver.Region;

-- Product
INSERT INTO gold.DimProduct (ProductKey, CategoryKey, ProductCode, ProductName)
SELECT ProductKey, CategoryKey, ProductCode, ProductName
FROM silver.Product;

-- Customer
INSERT INTO gold.DimCustomer (CustomerKey, RegionKey, CustomerCode, CustomerName)
SELECT CustomerKey, RegionKey, CustomerCode, CustomerName
FROM silver.Customer;

-- DimDate from Sales dates
;WITH AllDates AS (
    SELECT CAST(SalesDateTime AS date) AS DateVal
    FROM silver.Sales
)
INSERT INTO gold.DimDate (DateKey, [Date], [Day], [Month], MonthName, [Quarter], [Year], IsWeekend)
SELECT
    CONVERT(int, FORMAT(DateVal,'yyyyMMdd')) AS DateKey,
    DateVal,
    DAY(DateVal),
    MONTH(DateVal),
    DATENAME(MONTH, DateVal),
    DATEPART(QUARTER, DateVal),
    YEAR(DateVal),
    CASE WHEN DATEPART(WEEKDAY, DateVal) IN (1,7) THEN 1 ELSE 0 END
FROM AllDates
GROUP BY DateVal;

GO

/* ===== LOAD GOLD FACTS ===== */

INSERT INTO gold.FactSales (
    SalesKey,
    ProductKey,
    RegionKey,
    CustomerKey,
    DateKey,
    Quantity,
    UnitPrice,
    SalesAmount
)
SELECT
    s.SalesKey,
    s.ProductKey,
    s.RegionKey,
    s.CustomerKey,
    CONVERT(int, FORMAT(CAST(s.SalesDateTime AS date),'yyyyMMdd')) AS DateKey,
    s.Quantity,
    s.UnitPrice,
    s.Quantity * s.UnitPrice AS SalesAmount
FROM silver.Sales s;

GO

-- Validation: row counts
SELECT 'Product' AS Entity,
    (SELECT COUNT(*) FROM silver.Product) AS SilverCount,
    (SELECT COUNT(*) FROM gold.DimProduct) AS GoldCount
UNION ALL
SELECT 'Category',
    (SELECT COUNT(*) FROM silver.Category),
    (SELECT COUNT(*) FROM gold.DimCategory)
UNION ALL
SELECT 'Region',
    (SELECT COUNT(*) FROM silver.Region),
    (SELECT COUNT(*) FROM gold.DimRegion)
UNION ALL
SELECT 'Customer',
    (SELECT COUNT(*) FROM silver.Customer),
    (SELECT COUNT(*) FROM gold.DimCustomer)
UNION ALL
SELECT 'Sales',
    (SELECT COUNT(*) FROM silver.Sales),
    (SELECT COUNT(*) FROM gold.FactSales);
GO
```

05_pres_views.sql

```
/* 05_pres_views.sql
- Presentation views only (Power BI connects here)
- Uses CREATE OR ALTER for re-runability */

USE TeamProject_ProductPerformance;
GO

CREATE OR ALTER VIEW pres.vwDimDate
AS
SELECT
    DateKey,
    [Date],
    [Day],
    [Month],
    MonthName,
    [Quarter],
    [Year],
    IsWeekend
FROM gold.DimDate;
GO

CREATE OR ALTER VIEW pres.vwDimCategory
AS
SELECT
    CategoryKey,
    CategoryCode,
    CategoryName
FROM gold.DimCategory;
GO

CREATE OR ALTER VIEW pres.vwDimRegion
AS
SELECT
    RegionKey,
    RegionCode,
    RegionName
FROM gold.DimRegion;
GO

CREATE OR ALTER VIEW pres.vwDimProduct
AS
SELECT
    p.ProductKey,
    p.ProductCode,
    p.ProductName,
    c.CategoryKey,
    c.CategoryCode,
    c.CategoryName
FROM gold.DimProduct p
JOIN gold.DimCategory c ON c.CategoryKey = p.CategoryKey;
GO

CREATE OR ALTER VIEW pres.vwDimCustomer
AS
SELECT
    c.CustomerKey,
    c.CustomerCode,
    c.CustomerName,
    r.RegionKey,
    r.RegionCode,
    r.RegionName
FROM gold.DimCustomer c
JOIN gold.DimRegion r ON r.RegionKey = c.RegionKey;
GO

CREATE OR ALTER VIEW pres.vwFactSales
AS
SELECT
    f.SalesKey,
    f.ProductKey,
    dp.ProductCode,
    dp.ProductName,
    f.RegionKey,
    dr.RegionCode,
    dr.RegionName,
    f.CustomerKey,
    dc.CustomerCode,
    dc.CustomerName,
    d.[Date] AS SalesDate,
    f.Quantity,
    f.UnitPrice,
    f.SalesAmount
FROM gold.FactSales f
JOIN gold.DimProduct dp ON dp.ProductKey = f.ProductKey
JOIN gold.DimRegion dr ON dr.RegionKey = f.RegionKey
JOIN gold.DimCustomer dc ON dc.CustomerKey = f.CustomerKey
JOIN gold.DimDate d ON d.DateKey = f.DateKey;
GO

-- Validation: sample rows
SELECT TOP 5 * FROM pres.vwFactSales;
GO
```


06_readmission_metrics.sql

```
/* 06_readmission_metrics.sql (Product Performance Metrics)
- Implements metrics:
- Sales by product, category, region and year
- Units sold, revenue and average selling price
- Year-over-Year revenue growth
- Top and bottom performers via ranking
- Contribution to total revenue */

USE TeamProject_ProductPerformance;
GO

-- Re-runnable: summarise events
CREATE OR ALTER VIEW pres.vwProductPerformanceEvents
AS
WITH SalesAgg AS (
    SELECT
        dp.ProductKey,
        dp.ProductCode,
        dp.ProductName,
        dc.CategoryKey,
        dc.CategoryCode,
        dc.CategoryName,
        dr.RegionKey,
        dr.RegionCode,
        dr.RegionName,
        dd.[Year],
        SUM(fs.Quantity) AS UnitsSold,
        SUM(fs.SalesAmount) AS SalesRevenue,
        SUM(fs.SalesAmount) / NULLIF(SUM(fs.Quantity),0) AS AvgSellingPrice
    FROM gold.FactSales fs
    JOIN gold.DimProduct dp ON dp.ProductKey = fs.ProductKey
    JOIN gold.DimCategory dc ON dc.CategoryKey = dp.CategoryKey
    JOIN gold.DimRegion dr ON dr.RegionKey = fs.RegionKey
    JOIN gold.DimDate dd ON dd.DateKey = fs.DateKey
    GROUP BY
        dp.ProductKey, dp.ProductCode, dp.ProductName,
        dc.CategoryKey, dc.CategoryCode, dc.CategoryName,
        dr.RegionKey, dr.RegionCode, dr.RegionName,
        dd.[Year]
),
Growth AS (
    SELECT
        sa.*,
        LAG(sa.SalesRevenue) OVER (PARTITION BY sa.ProductKey ORDER BY sa.[Year]) AS PrevYearRevenue,
        CASE
            WHEN LAG(sa.SalesRevenue) OVER (PARTITION BY sa.ProductKey ORDER BY sa.[Year]) IS NULL
            OR LAG(sa.SalesRevenue) OVER (PARTITION BY sa.ProductKey ORDER BY sa.[Year]) = 0
            THEN NULL
            ELSE (sa.SalesRevenue - LAG(sa.SalesRevenue) OVER (PARTITION BY sa.ProductKey ORDER BY sa.[Year]))
            / LAG(sa.SalesRevenue) OVER (PARTITION BY sa.ProductKey ORDER BY sa.[Year]) * 100
        END AS YoYGrowthPct
    FROM SalesAgg sa
),
Ranked AS (
    SELECT
        g.*,
        ROW_NUMBER() OVER (PARTITION BY g.[Year] ORDER BY g.SalesRevenue DESC) AS RankByRevenueDesc,
        ROW_NUMBER() OVER (PARTITION BY g.[Year] ORDER BY g.SalesRevenue ASC) AS RankByRevenueAsc,
        g.SalesRevenue / SUM(g.SalesRevenue) OVER (PARTITION BY g.[Year]) * 100 AS ContributionPct
    FROM Growth g
)
SELECT * FROM Ranked;
GO

-- KPI summary view
CREATE OR ALTER VIEW pres.vwProductPerformanceKPIs
AS
WITH Totals AS (
    SELECT
        SUM(fs.Quantity) AS TotalUnits,
        SUM(fs.SalesAmount) AS TotalRevenue,
        SUM(fs.SalesAmount) / NULLIF(SUM(fs.Quantity),0) AS AvgUnitPrice
    FROM gold.FactSales fs
),
Yearly AS (
    SELECT
        dd.[Year],
        SUM(fs.SalesAmount) AS YearRevenue,
        SUM(fs.Quantity) AS YearUnits
    FROM gold.FactSales fs
    JOIN gold.DimDate dd ON dd.DateKey = fs.DateKey
    GROUP BY dd.[Year]
),
YearlyGrowth AS (
    SELECT
        y.[Year],
        y.YearRevenue,
        y.YearUnits,
        LAG(y.YearRevenue) OVER (ORDER BY y.[Year]) AS PrevRevenue,
        CASE
            WHEN LAG(y.YearRevenue) OVER (ORDER BY y.[Year]) IS NULL
            OR LAG(y.YearRevenue) OVER (ORDER BY y.[Year]) = 0
            THEN NULL
            ELSE (y.YearRevenue - LAG(y.YearRevenue) OVER (ORDER BY y.[Year]))
            / LAG(y.YearRevenue) OVER (ORDER BY y.[Year]) * 100
        END AS YoYGrowthPct
    FROM Yearly y
)
SELECT
    t.TotalUnits,
    t.TotalRevenue,
    t.AvgUnitPrice,
    yg.[Year],
    yg.YearRevenue,
    yg.YearUnits,
    yg.PrevRevenue,
    yg.YoYGrowthPct
FROM Totals t
CROSS JOIN YearlyGrowth yg;
GO

-- Validation: sample KPIs
SELECT * FROM pres.vwProductPerformanceKPIs;
GO
```


07_etl_stored_procs.sql

```
/* 07_etl_stored_procs.sql
- usp_LoadSilver_All: truncates + reloads Silver using the same set-based logic as Script 03
- usp_LoadGold_All: truncates + reloads Gold using the same set-based logic as Script 04
- usp_RunDQ_Checks: key checks + logging
- usp_FullRefresh: runs all */

USE TeamProject_ProductPerformance;
GO

CREATE OR ALTER PROCEDURE etl.usp_LoadSilver_All
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;

    BEGIN TRY
        BEGIN TRAN;
        -- Truncate in dependency order
        DELETE FROM silver.Sales;
        DELETE FROM silver.Customer;
        DELETE FROM silver.Product;
        DELETE FROM silver.Category;
        DELETE FROM silver.Region;
        -- Categories
        INSERT INTO silver.Category (CategoryCode, CategoryName)
        SELECT DISTINCT
            CategoryCode,
            CONCAT(UPPER(LEFT(LTRIM(RTRIM(CategoryName)),1)),
            LOWER(SUBSTRING(LTRIM(RTRIM(CategoryName)),2,200)))
        FROM bronze.Category;
        -- Maps
        IF OBJECT_ID('tempdb..#CategoryMap') IS NOT NULL DROP TABLE #CategoryMap;
        SELECT b.CategoryID, s.CategoryKey
        INTO #CategoryMap
        FROM bronze.Category b
        JOIN silver.Category s ON s.CategoryCode = b.CategoryCode;
        -- Regions
        INSERT INTO silver.Region (RegionCode, RegionName)
        SELECT DISTINCT
            RegionCode,
            CONCAT(UPPER(LEFT(LTRIM(RTRIM(RegionName)),1)),
            LOWER(SUBSTRING(LTRIM(RTRIM(RegionName)),2,200)))
        FROM bronze.Region;
        IF OBJECT_ID('tempdb..#RegionMap') IS NOT NULL DROP TABLE #RegionMap;
        SELECT b.RegionID, s.RegionKey
        INTO #RegionMap
        FROM bronze.Region b
        JOIN silver.Region s ON s.RegionCode = b.RegionCode;
        -- Products
        ;WITH SrcProd AS (
        SELECT
            b.ProductID,
            b.CategoryID,
            b.ProductCode,
            b.ProductName,
            ROW_NUMBER() OVER (PARTITION BY b.ProductCode ORDER BY b.ProductID) AS rn
        FROM bronze.Product b
        )
        INSERT INTO silver.Product (CategoryKey, ProductCode, ProductName)
        SELECT
            cm.CategoryKey,
            sp.ProductCode,
            CONCAT(UPPER(LEFT(LTRIM(RTRIM(sp.ProductName)),1)),
            LOWER(SUBSTRING(LTRIM(RTRIM(sp.ProductName)),2,200)))
        FROM SrcProd sp
        JOIN #CategoryMap cm ON cm.CategoryID = sp.CategoryID
        WHERE sp.rn = 1;
        IF OBJECT_ID('tempdb..#ProductMap') IS NOT NULL DROP TABLE #ProductMap;
        SELECT b.ProductID, s.ProductKey
        INTO #ProductMap
        FROM bronze.Product b
        JOIN silver.Product s ON s.ProductCode = b.ProductCode;
        -- Customers
        ;WITH SrcCust AS (
        SELECT
            b.CustomerID,
            b.RegionID,
            b.CustomerCode,
            b.CustomerName,
            ROW_NUMBER() OVER (PARTITION BY b.CustomerCode ORDER BY b.CustomerID) AS rn
        FROM bronze.Customer b
        )
        INSERT INTO silver.Customer (RegionKey, CustomerCode, CustomerName)
        SELECT
            rm.RegionKey,
            sc.CustomerCode,
            CONCAT(UPPER(LEFT(LTRIM(RTRIM(sc.CustomerName)),1)),
            LOWER(SUBSTRING(LTRIM(RTRIM(sc.CustomerName)),2,200)))
        FROM SrcCust sc
        JOIN #RegionMap rm ON rm.RegionID = sc.RegionID
        WHERE sc.rn = 1;
        IF OBJECT_ID('tempdb..#CustomerMap') IS NOT NULL DROP TABLE #CustomerMap;
        SELECT b.CustomerID, s.CustomerKey
        INTO #CustomerMap
        FROM bronze.Customer b
        JOIN silver.Customer s ON s.CustomerCode = b.CustomerCode;
        -- Sales
        INSERT INTO silver.Sales (
            BronzeSalesID,
            ProductKey,
            RegionKey,
            CustomerKey,
            SalesDateTime,
            Quantity,
            UnitPrice
        )
        SELECT
            bs.SalesID,
            pm.ProductKey,
            rm.RegionKey,
            cm.CustomerKey,
            bs.SalesDateTime,
            CASE WHEN bs.Quantity <= 0 THEN 0 ELSE bs.Quantity END,
            CASE WHEN bs.UnitPrice <= 0 THEN 0.00 ELSE bs.UnitPrice END
        FROM bronze.Sales bs
        JOIN #ProductMap pm ON pm.ProductID = bs.ProductID
        JOIN #RegionMap rm ON rm.RegionID = bs.RegionID
        JOIN #CustomerMap cm ON cm.CustomerID = bs.CustomerID;
        COMMIT TRAN;
        INSERT INTO dq.DataQualityLog (TableName, CheckName, CheckResult, ErrorCount)
        VALUES ('etl.usp_LoadSilver_All','Load completed','Success',0);
        END TRY
        BEGIN CATCH
            IF @@TRANCOUNT > 0 ROLLBACK TRAN;
            INSERT INTO dq.DataQualityLog (TableName, CheckName, CheckResult, ErrorCount)
            VALUES ('etl.usp_LoadSilver_All','Load failed', ERROR_MESSAGE(), 1);
            THROW;
        END CATCH

    END
GO

CREATE OR ALTER PROCEDURE etl.usp_LoadGold_All
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;

    BEGIN TRY
        BEGIN TRAN;
        -- Truncate fact first
        DELETE FROM gold.FactSales;
        -- Truncate dims
        DELETE FROM gold.DimCustomer;
        DELETE FROM gold.DimRegion;
        DELETE FROM gold.DimProduct;
        DELETE FROM gold.DimCategory;
        DELETE FROM gold.DimDate;
        -- Reload dims
        INSERT INTO gold.DimCategory (CategoryKey, CategoryCode, CategoryName)
        SELECT CategoryKey, CategoryCode, CategoryName
        FROM silver.Category;
        INSERT INTO gold.DimRegion (RegionKey, RegionCode, RegionName)
        SELECT RegionKey, RegionCode, RegionName
        FROM silver.Region;
        INSERT INTO gold.DimProduct (ProductKey, CategoryKey, ProductCode, ProductName)
        SELECT ProductKey, CategoryKey, ProductCode, ProductName
        FROM silver.Product;
        INSERT INTO gold.DimCustomer (CustomerKey, RegionKey, CustomerCode, CustomerName)
        SELECT CustomerKey, RegionKey, CustomerCode, CustomerName
        FROM silver.Customer;
        -- Reload DimDate
        ;WITH AllDates AS (
        SELECT CAST(SalesDateTime AS date) AS DateVal
        FROM silver.Sales
        )
        INSERT INTO gold.DimDate (DateKey, [Date], [Day], [Month], MonthName, [Quarter], [Year], IsWeekend)
        SELECT
            CONVERT(int, FORMAT(DateVal,'yyyyMMdd')),
            DateVal,
            DAY(DateVal),
            MONTH(DateVal),
            DATENAME(MONTH, DateVal),
            DATEPART(QUARTER, DateVal),
            YEAR(DateVal),
            CASE WHEN DATEPART(WEEKDAY, DateVal) IN (1,7) THEN 1 ELSE 0 END
        FROM AllDates
        GROUP BY DateVal;
        -- Reload Fact
        INSERT INTO gold.FactSales (
            SalesKey,
            ProductKey,
            RegionKey,
            CustomerKey,
            DateKey,
            Quantity,
            UnitPrice,
            SalesAmount
        )
        SELECT
            s.SalesKey,
            s.ProductKey,
            s.RegionKey,
            s.CustomerKey,
            CONVERT(int, FORMAT(CAST(s.SalesDateTime AS date),'yyyyMMdd')),
            s.Quantity,
            s.UnitPrice,
            s.Quantity * s.UnitPrice
        FROM silver.Sales s;
        COMMIT TRAN;
        INSERT INTO dq.DataQualityLog (TableName, CheckName, CheckResult, ErrorCount)
        VALUES ('etl.usp_LoadGold_All','Load completed','Success',0);
        END TRY
        BEGIN CATCH
            IF @@TRANCOUNT > 0 ROLLBACK TRAN;
            INSERT INTO dq.DataQualityLog (TableName, CheckName, CheckResult, ErrorCount)
            VALUES ('etl.usp_LoadGold_All','Load failed', ERROR_MESSAGE(), 1);
            THROW;
        END CATCH

    END
GO

CREATE OR ALTER PROCEDURE etl.usp_RunDQ_Checks
AS
BEGIN
    SET NOCOUNT ON;

    -- Negative or zero quantities
    DECLARE @BadQty INT = (SELECT COUNT(*) FROM silver.Sales WHERE Quantity <= 0);
    INSERT INTO dq.DataQualityLog (TableName, CheckName, CheckResult, ErrorCount)
    VALUES ('silver.Sales','Quantity > 0',
    CASE WHEN @BadQty = 0 THEN 'Pass' ELSE 'Fail' END, @BadQty);
    -- Negative or zero unit prices
    DECLARE @BadPrice INT = (SELECT COUNT(*) FROM silver.Sales WHERE UnitPrice <= 0);
    INSERT INTO dq.DataQualityLog (TableName, CheckName, CheckResult, ErrorCount)
    VALUES ('silver.Sales','UnitPrice > 0',
    CASE WHEN @BadPrice = 0 THEN 'Pass' ELSE 'Fail' END, @BadPrice);
    -- Orphan sales (should not occur after mapping)
    DECLARE @OrphanSales INT = (
    SELECT COUNT(*)
    FROM silver.Sales s
    LEFT JOIN silver.Product p ON p.ProductKey = s.ProductKey
    LEFT JOIN silver.Region r ON r.RegionKey = s.RegionKey
    LEFT JOIN silver.Customer c ON c.CustomerKey = s.CustomerKey
    WHERE p.ProductKey IS NULL OR r.RegionKey IS NULL OR c.CustomerKey IS NULL
    );
    INSERT INTO dq.DataQualityLog (TableName, CheckName, CheckResult, ErrorCount)
    VALUES ('silver.Sales','No orphan references',
    CASE WHEN @OrphanSales = 0 THEN 'Pass' ELSE 'Fail' END, @OrphanSales);

    END
GO

CREATE OR ALTER PROCEDURE etl.usp_FullRefresh
AS
BEGIN
    SET NOCOUNT ON;
    EXEC etl.usp_LoadSilver_All;
    EXEC etl.usp_LoadGold_All;
    EXEC etl.usp_RunDQ_Checks;
    END
GO
```


08_validation_reconciliation.sql

```
/* 08_validation_reconciliation.sql
- Reconciles row counts across layers
- Orphan checks
- Sales sanity checks
- Top and bottom products */

USE TeamProject_ProductPerformance;
GO

-- Row counts across layers
SELECT 'Category' AS Entity,
(SELECT COUNT(*) FROM bronze.Category) AS BronzeCount,
(SELECT COUNT(*) FROM silver.Category) AS SilverCount,
(SELECT COUNT(*) FROM gold.DimCategory) AS GoldCount;

SELECT 'Product' AS Entity,
(SELECT COUNT(*) FROM bronze.Product) AS BronzeCount,
(SELECT COUNT(*) FROM silver.Product) AS SilverCount,
(SELECT COUNT(*) FROM gold.DimProduct) AS GoldCount;

SELECT 'Region' AS Entity,
(SELECT COUNT(*) FROM bronze.Region) AS BronzeCount,
(SELECT COUNT(*) FROM silver.Region) AS SilverCount,
(SELECT COUNT(*) FROM gold.DimRegion) AS GoldCount;

SELECT 'Customer' AS Entity,
(SELECT COUNT(*) FROM bronze.Customer) AS BronzeCount,
(SELECT COUNT(*) FROM silver.Customer) AS SilverCount,
(SELECT COUNT(*) FROM gold.DimCustomer) AS GoldCount;

SELECT 'Sales' AS Entity,
(SELECT COUNT(*) FROM bronze.Sales) AS BronzeCount,
(SELECT COUNT(*) FROM silver.Sales) AS SilverCount,
(SELECT COUNT(*) FROM gold.FactSales) AS GoldCount;

-- Orphan checks (should be zero)
SELECT COUNT(*) AS OrphanSales
FROM silver.Sales s
LEFT JOIN silver.Product p ON p.ProductKey = s.ProductKey
LEFT JOIN silver.Region r ON r.RegionKey = s.RegionKey
LEFT JOIN silver.Customer c ON c.CustomerKey = s.CustomerKey
WHERE p.ProductKey IS NULL OR r.RegionKey IS NULL OR c.CustomerKey IS NULL;

-- Sales statistics after cleaning
SELECT
    MIN(UnitPrice) AS MinUnitPrice,
    MAX(UnitPrice) AS MaxUnitPrice,
    AVG(CAST(UnitPrice AS decimal(18,2))) AS AvgUnitPrice,
    MIN(Quantity) AS MinQuantity,
    MAX(Quantity) AS MaxQuantity,
    AVG(CAST(Quantity AS decimal(18,2))) AS AvgQuantity
FROM silver.Sales;

-- Total revenue and units in Gold
SELECT
    SUM(SalesAmount) AS TotalRevenue,
    SUM(Quantity) AS TotalUnits
FROM gold.FactSales;

-- Top 5 products by revenue (overall)
SELECT TOP 5
    dp.ProductCode,
    dp.ProductName,
    SUM(fs.SalesAmount) AS Revenue
FROM gold.FactSales fs
JOIN gold.DimProduct dp ON dp.ProductKey = fs.ProductKey
GROUP BY dp.ProductCode, dp.ProductName
ORDER BY Revenue DESC;

-- Bottom 5 products by revenue (overall)
SELECT TOP 5
    dp.ProductCode,
    dp.ProductName,
    SUM(fs.SalesAmount) AS Revenue
FROM gold.FactSales fs
JOIN gold.DimProduct dp ON dp.ProductKey = fs.ProductKey
GROUP BY dp.ProductCode, dp.ProductName
ORDER BY Revenue ASC;

GO
```


09_training_sql_fundamentals.sql

```
/* 09_training_sql_fundamentals.sql
- DDL, DML, DQL, TCL examples
- Safe: uses dbo.SampleSales only */

USE TeamProject_ProductPerformance;
GO

IF OBJECT_ID('dbo.SampleSales','U') IS NOT NULL DROP TABLE dbo.SampleSales;
GO

-- DDL
CREATE TABLE dbo.SampleSales (
    SaleID INT IDENTITY(1,1) PRIMARY KEY,
    Product NVARCHAR(50) NOT NULL,
    Quantity INT NOT NULL,
    UnitPrice DECIMAL(10,2) NOT NULL
);

-- DML
INSERT INTO dbo.SampleSales (Product, Quantity, UnitPrice)
VALUES
    ('Widget', 10, 5.00),
    ('Gadget', 20, 7.50),
    ('Thingamajig', 5, 12.00);

-- DQL + GROUP BY + HAVING
SELECT
    Product,
    SUM(Quantity) AS TotalQty,
    SUM(Quantity * UnitPrice) AS TotalRevenue
FROM dbo.SampleSales
GROUP BY Product
HAVING SUM(Quantity) >= 5;

-- Window functions
SELECT
    SaleID,
    Product,
    Quantity,
    UnitPrice,
    ROW_NUMBER() OVER (ORDER BY SaleID) AS RowNum,
    AVG(UnitPrice) OVER () AS AvgPrice
FROM dbo.SampleSales;

-- UPDATE / DELETE
UPDATE dbo.SampleSales
SET UnitPrice = 8.00
WHERE Product = 'Widget';

DELETE FROM dbo.SampleSales
WHERE Product = 'Thingamajig';

-- TCL with TRY/CATCH
BEGIN TRY
    BEGIN TRAN;
    INSERT INTO dbo.SampleSales (Product, Quantity, UnitPrice)
    VALUES ('NewItem', 2, 3.50);

    -- Simulate an error
    DECLARE @x INT = 1/0;
    COMMIT TRAN;

END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK TRAN;
    PRINT 'Error occurred: ' + ERROR_MESSAGE();
END CATCH;

DROP TABLE dbo.SampleSales;
GO
```

Key Implementation Features



Medallion Architecture

Complete implementation of Bronze, Silver and Gold layers with proper separation of concerns. Raw data preservation, cleansing and transformation, and analytics-optimised star schema.



ETL Automation

Stored procedures for repeatable ETL execution. Full refresh capability, error handling with rollback, and logging of all operations to the data quality log.



Presentation Layer

Clean views for Power BI consumption. Decoupled from underlying tables, re-runnable with CREATE OR ALTER, and optimised for reporting performance.



Data Quality

Comprehensive data quality logging and validation. Detects and corrects invalid quantities and prices, identifies orphan records, and provides reconciliation across all layers.



Performance Metrics

Advanced analytics using window functions. Year-over-year growth calculations, product rankings, contribution analysis, and comprehensive KPI views for business intelligence.



Training Materials

Self-contained SQL fundamentals script. Demonstrates DDL, DML, DQL, window functions and transaction control without affecting project tables.